

Intrinsic Islands: An Oracle-Anchored Escape Hatch for a Portable Kernel

Hanzo AI — ML Systems

hanzo-kernel Paper 07.3 — part of the *One Source, Every Backend* series

Abstract

Every portable kernel DSL eventually meets a loop that wants a target-specific instruction: the hardware 4-wide int8 dot on one backend, a subgroup shuffle on another. The tempting response is to fork a whole per-backend kernel for that one line — which rebuilds, in miniature, the very Cartesian product the DSL exists to collapse. This paper presents a disciplined alternative: the *intrinsic island*, a scoped, target-gated block written *inside* the one kernel. The kernel keeps one signature, one launch contract, one reference, and one test; only the marked inner expression differs per target. The design rests on two rules. First, selection is a *compile-time match* on a target tag the launch layer fills in and the caller never sees, so exactly one arm is lowered per backend and nothing else compiles. Second, the `default` arm is *normative*: it *is* the reference semantics, the CPU always takes it, and every accelerated arm must equal it byte-for-byte on the CPU oracle before it can ship. An island is thus an escape hatch that cannot escape correctness — the accelerated instruction is never trusted, only proven equivalent. We show the mechanism, the one-kernel-one-oracle test, and why an island beats a fork on exactly the axis that matters.

1 The pain: one line wants a special instruction

A quantized matvec spends its inner loop on a 4-wide integer dot product: four `int8` weights times four `int8` activations, summed to an `i32`. On NVIDIA, AMD, Apple, and Vulkan hardware this is a *single instruction* (`dp4a` / `OpSDotAccSat`) — an order of magnitude cheaper than four scalar multiply-adds. A portable kernel that emits four scalar MACs leaves that instruction on the table; one that emits the hardware dot cannot run on a target without it (the CPU reference has no such op).

The reflex is to fork: a hand-written `matvec.cuda` that uses the intrinsic, and a portable `matvec.fallback` that does not. But a fork for one inner line drags the entire kernel with it — its signature, its launch bounds, its indexing, its reference, its test — all now duplicated and free to drift. That is the Cartesian product (backends $\times$ ops) reappearing at the granularity of a single expression. We want the hardware instruction where it exists and the portable arithmetic where it does not, *without* a second copy of the kernel.

2 The idea: gate the expression, not the kernel

An island moves the fork inward, from the whole kernel to the one expression that actually differs (Listing 1). The kernel is written once. At the one spot that needs a target-specific idiom, an

`island!` block lists accelerated arms by backend and a mandatory `default` arm. The rest of the kernel — the loop, the indexing, the accumulation, the bounds — is shared verbatim.

Listing 1: An island in a `dp4a` matvec. The accelerated arm issues the hardware 4-wide int8 dot; the `default` arm — the normative oracle, taken by the CPU and any unlisted target — computes the identical contraction with portable scalar lane MACs. Same math, two idioms.

```
let dp = island! {
  // Accelerated: the hardware 4-wide int8 dot (dp4a / OpSDot), one instruction.
  cuda | rocm | metal | vulkan => { w.dot(x) }
  // NORMATIVE oracle: the identical integer contraction, portable scalar lane MACs.
  default => { w[0]*x[0] + w[1]*x[1] + w[2]*x[2] + w[3]*x[3] }
};
```

Two properties make this safe rather than merely convenient, and both are structural.

The branch site names backends, not a target. The author writes `cuda | rocm | metal | vulkan => ...`; they never write “if I am on CUDA.” The *tag* that selects the arm is filled in elsewhere (§3), so a kernel author cannot accidentally couple the math to a runtime string.

default is the definition. The accelerated arms are optimizations *of* the `default` arm. `default` is not a fallback in the sense of “lower quality if unlucky” — it is the *normative* semantics that the accelerated arms must reproduce. This is the inversion that makes an island trustworthy: the portable arithmetic is the truth, and the fast instruction has to match it.

3 The mechanism: target selection at compile time

The subtlety is that the lowering engine compiles the same target-agnostic IR for every backend; at kernel-expansion time there is no “which backend am I” string available inside the kernel, only a capability set. But the backend brand *is* known on the *host*, from the runtime’s name (“cpu”, “cuda”, “hip”, “wgpu<spirv>”, “wgpu<msl>”). So an island is selected by a compile-time tag threaded in from the host (Listing 2):

Listing 2: The target tag is derived on the host from the runtime name and threaded into the kernel as a `#[comptime]` parameter. The `#[kernel]` macro rewrites `island!{...}` into a comptime `match target`. Unknown names fall to `Cpu` — fail safe, never a wrong accelerated arm.

```
pub enum Target { Cpu, Cuda, Rocm, Metal, Vulkan, WebGpu } // #[comptime] tag: Eq + Hash + Copy

pub fn from_runtime_name(name: &str) -> Target {
  match name {
    "cuda" => Target::Cuda,
    "hip" => Target::Rocm,
    n if n.contains("spirv") => Target::Vulkan,
    n if n.contains("msl") => Target::Metal,
    n if n.starts_with("wgpu") => Target::WebGpu,
    _ => Target::Cpu, // fail safe -> the oracle arm
  }
}
```

Because `target` is a `#[comptime]` value, the `match` it drives is a comptime match over a constant scrutinee: **only the selected arm is lowered**. Every backend compiles exactly its own arm and nothing else — there is no runtime branch, no dead code for other targets in the shipped kernel.

The tag is kept out of the authoring surface twice over: the branch site lists backends (not a target), and the host launch wrapper fills the tag (`Target::of::<R>`), so callers pass only data. One signature, one launch, every backend.

4 The oracle contract, tested

Here is the rule that makes an island an escape hatch that cannot escape correctness. The CPU runtime is *always* tagged `Target::Cpu` — a variant no accelerated arm ever claims — so on the CPU every island resolves to `default`. The CPU runtime is therefore the single bit-exact oracle for every island (the general form of this law, and its cross-backend enforcement, is the companion oracle paper, [5]). The test states the contract directly and mechanically: run the kernel on the CPU with each accelerated tag *forced*, and it must equal the `default` result byte-for-byte (Listing 3).

Listing 3: The oracle-equivalence test. Every accelerated arm, executed on the CPU runtime with its tag forced, must be byte-identical to the `default` arm and to the plain-Rust reference. An accelerated instruction is never trusted — it is proven equal to the normative arm.

```
// The default arm (CPU is tagged Cpu -> default) must equal the plain-Rust oracle.
let default = matvec_island_run_with::<CpuRuntime>(&c, &wq, &xq, &wd, rows, k, Target::Cpu);
assert_eq!(bits(&default), bits(&matvec_island_ref(&wq, &xq, &wd, rows, k)));

// Every accelerated arm, forced onto the CPU runtime, must equal the oracle byte-for-byte.
for accel in [Target::Cuda, Target::Rocm, Target::Metal, Target::Vulkan, Target::WebGpu] {
    let out = matvec_island_run_with::<CpuRuntime>(&c, &wq, &xq, &wd, rows, k, accel);
    assert_eq!(bits(&out), bits(&default), "island arm {accel:?} diverged from default");
}
```

A separate test asserts the *production* launch — the one that derives the tag from the runtime and hides it — resolves to the oracle arm on the CPU without the caller naming a target. So the property is proven both under forced tags (every arm equals `default`) and under the real launch path (the caller-facing API does the right thing by default).

5 Why an island beats a fork

The comparison is not aesthetic; it is about what can drift and what can be verified (Table 1). A fork gives two kernels: two signatures to keep in sync, two references to trust, two tests to keep honest, and no mechanical guarantee that the fast kernel computes what the portable one does — the equivalence lives only in a human’s intention. An island gives one kernel: one signature, one reference (the `default` arm), one test that *proves* every accelerated arm equals it. The accelerated instruction is contained, not trusted.

	Fork (per-backend kernel)	Island (scoped block)
Signatures	one per backend, drift-prone	one, shared
Reference	one per fork, trust-based	one <code>default</code> arm, normative
Equivalence	human intention only	proven byte-for-byte on the CPU oracle
What differs	the whole kernel	the one gated expression
New backend	another full kernel	another arm, or falls to <code>default</code>

Table 1: An island localizes the difference to the expression that actually differs and anchors it to a CPU-checkable normative arm. A fork localizes nothing and anchors nothing.

6 The designed seam

An island is not a licence to sprinkle intrinsics; it is the *one* sanctioned place a per-backend idiom may live, precisely because it comes with the oracle contract attached. That matters most for the operation this series shows does *not* collapse into portable form: block-quantized tensor-core MMQ, whose viable implementation is per-backend by nature ([6]). Even there, an island holds the hand-written tensor-core arm behind a normative, CPU-checkable `default`, so the one uncollapsed kernel still keeps one signature, one oracle, one test. The escape hatch exists so that the rare irreducibly-per-backend kernel does not force a return to forks — it is the boundary condition of the whole “one source, every backend” program, made safe.

References

- [1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL*. Paper 07 (umbrella).
- [2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning*. Paper 07.2.
- [4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch*. Paper 07.3 (this paper).
- [5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4.
- [6] Hanzo AI. *When the Product Won't Collapse: An int8 Tensor-Core MMQ Negative Result*. Paper 07.5.
- [7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product*. Paper 07.6.